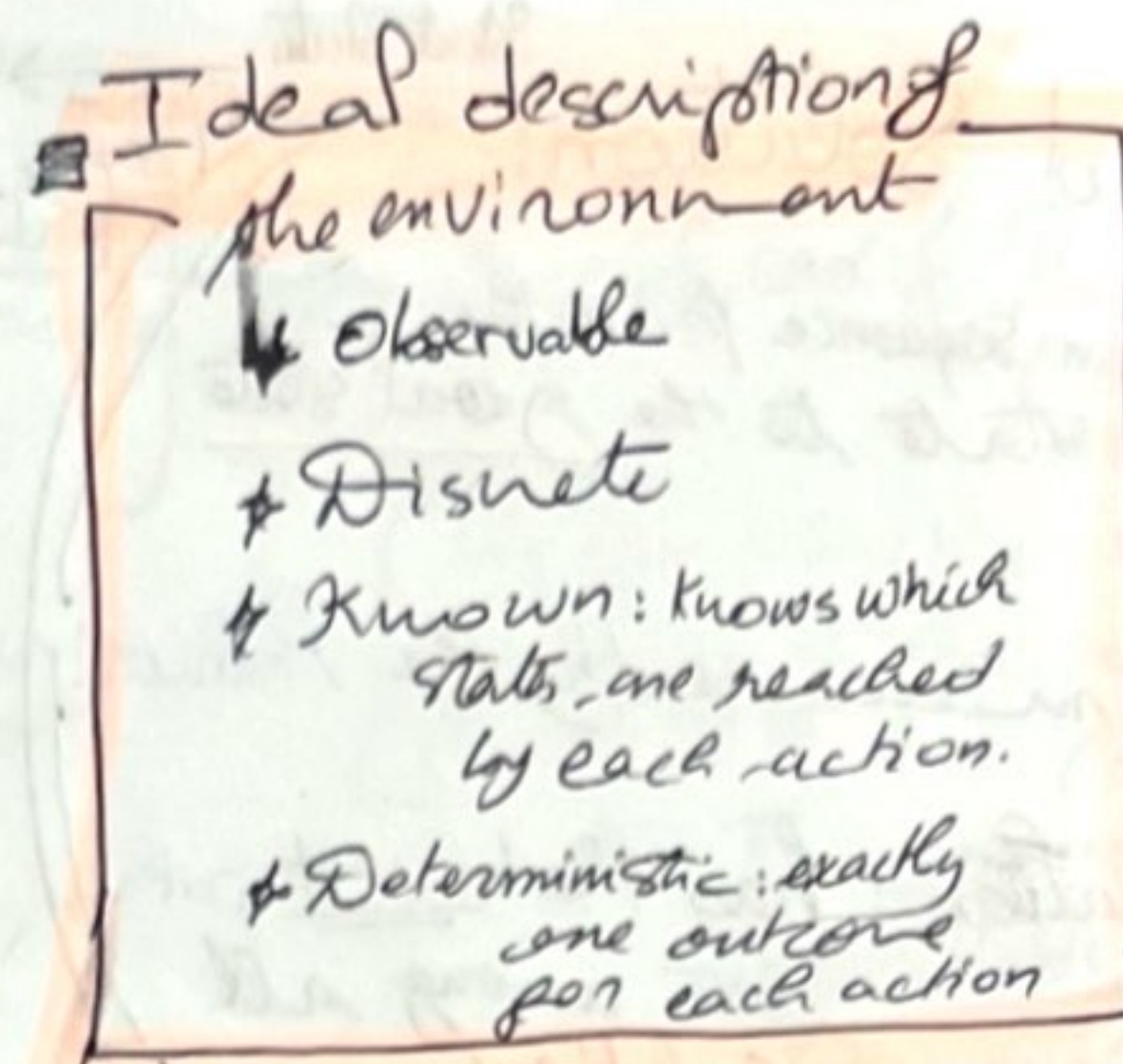
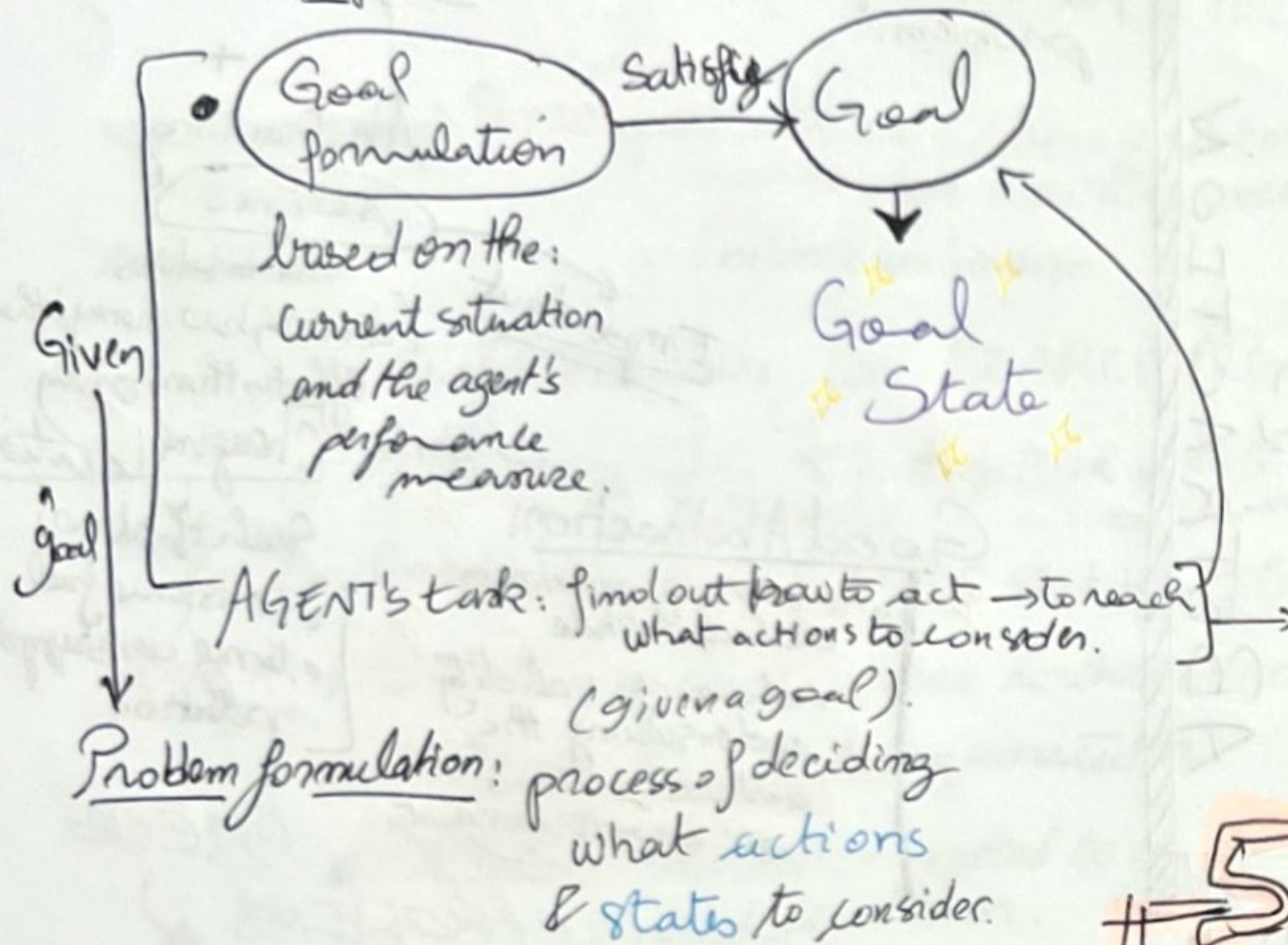


I - Solving Problems:

Step 01



Unknown WORLD
choose action **RANDOMLY**

We put those 5 in Data Structures as input to a problem solving algo.

Well defined definition of a problem

①



implicitly define

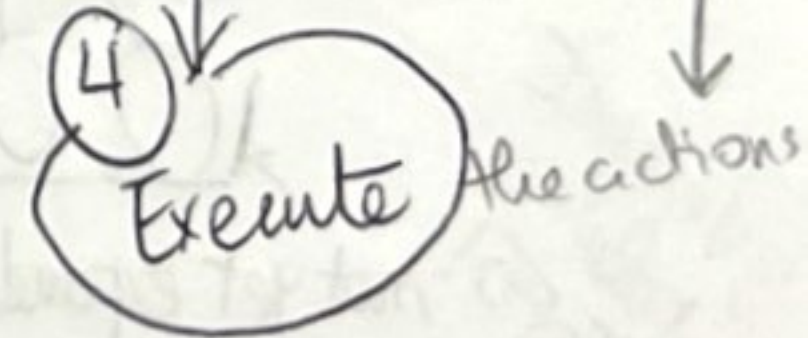
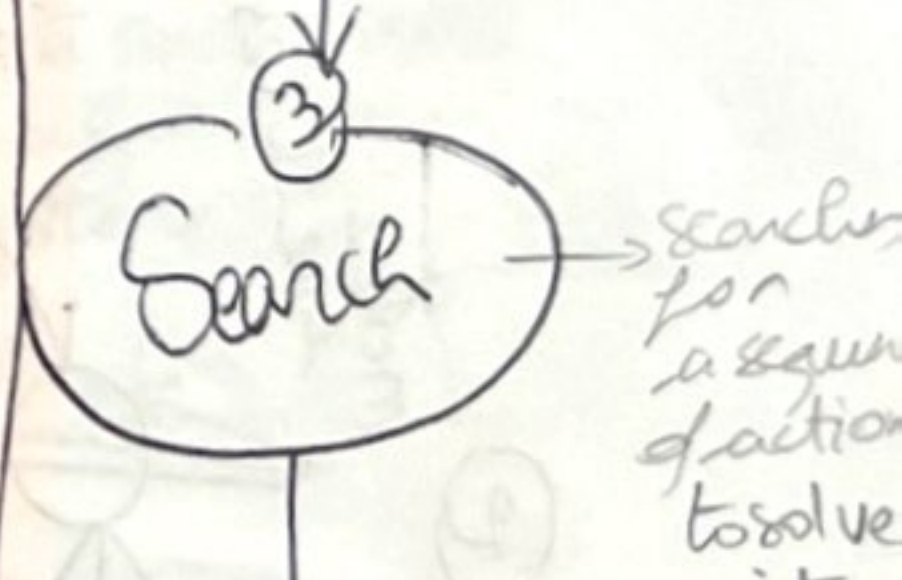
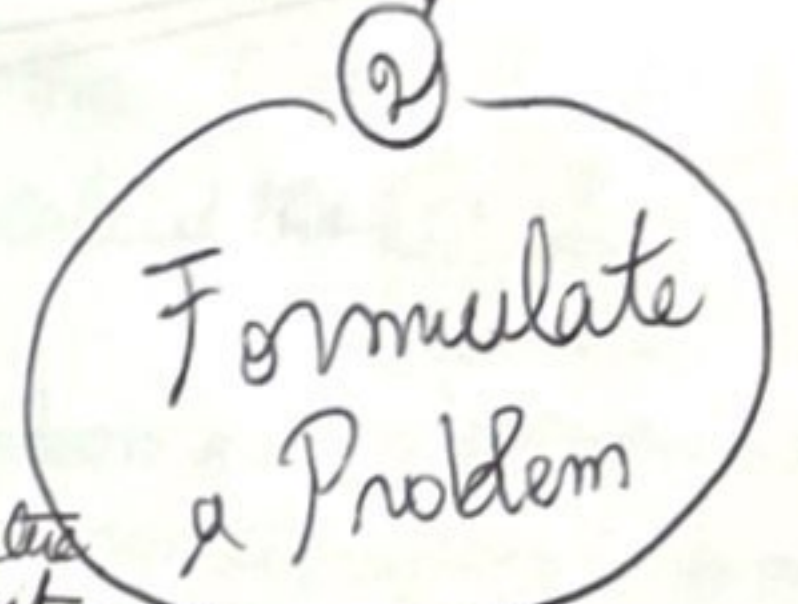
- ★ **INITIAL STATE**
- ★ **ALL POSSIBLE ACTIONS** available in a given state
- ★ **TRANSITION MODEL** description of what each action does.
- ★ **GOAL TEST** WHETHER a given state is the goal or no.
- ★ **PATH COST** function that assigns numeric cost to each path, reflects performance measure.

Step cost of taking action a in states to reach S' : $c(a, S, S')$

cost of path = $\sum$ [cost of individual actions along the path]

intends to capture action cost

Future S.



Well defined Solution:

- Solution: action sequence from the initial state to the goal state

↓
S. quality: measured by the Path Cost

Cost ↓
Optimal solution: has the lowest path Cost among all solutions.



Formulating a problem:

⚠ Abstraction: removing details from the representation of the problem.

State description
Abstraction.

ABSTRACTION

+
Abstracting Actions themselves.

Important
Considered ← Changing location!
(driving has many other effects than only such that as:)

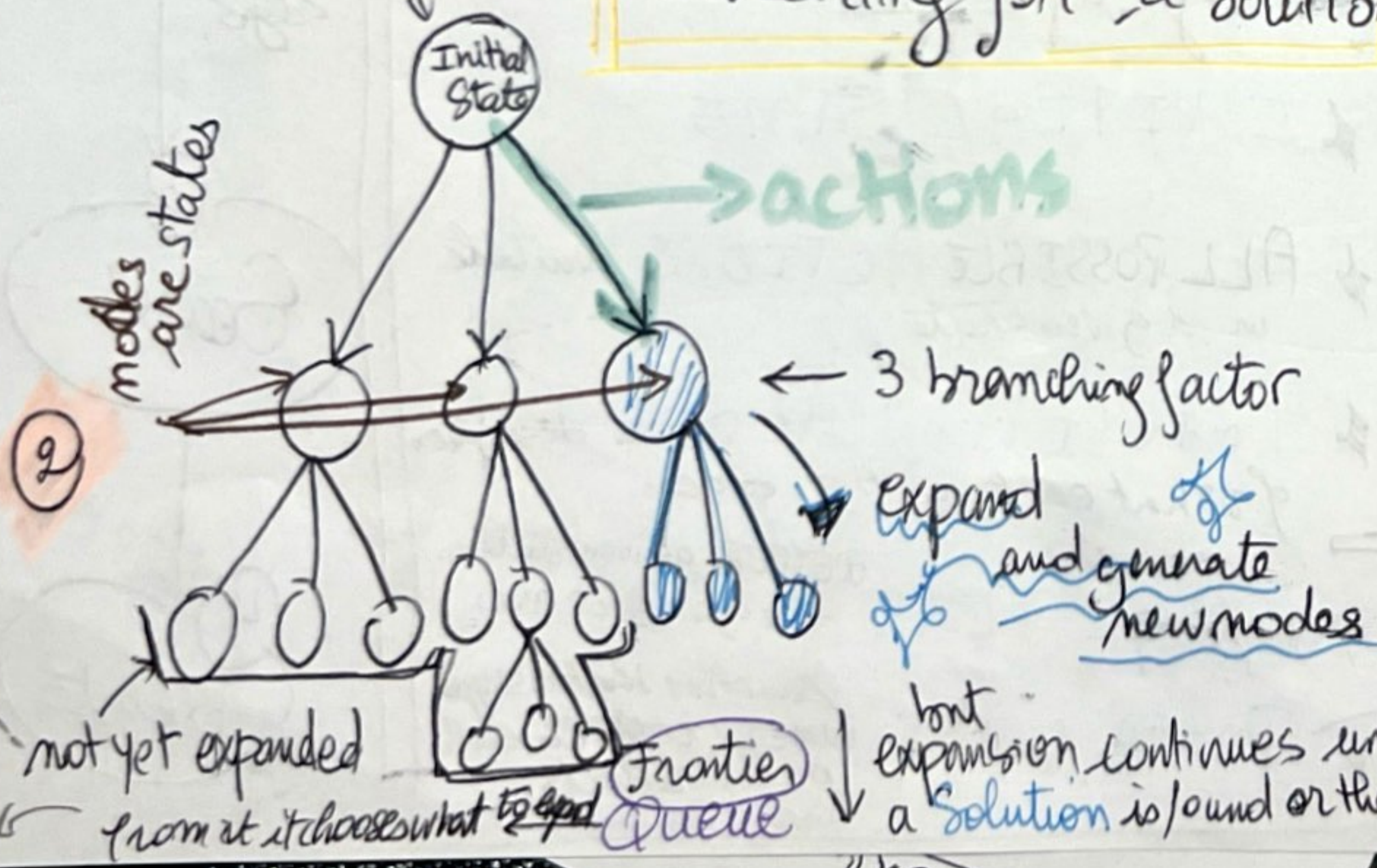
Good Abstraction:
• removing maximum details while retaining validity and ensuring the abstract actions are easy to carry out.

[Consume fuel
time consumption
pollution]

check

(EXAMPLE PROBLEMS)
PAGE (slide 14)

Searching for a Solution



• Search tree: possible action sequences starting at the initial state

• Key decision: which child to expand? / makes difference between search algorithms
↓
select a child and keep others as alternatives (if chosen doesn't get to goal).

SEARCH STRATEGIES vary depending on Which Node to EXPAND.

LOOPY PATH: Contains a repeated state (already explored) search tree for Romania is INFINITE!
we can visit again the same nodes

→ Since path costs are additive and cost is non-negative then we can avoid falling in loops.

Infrastructure for SEARCH ALGO.

• requires a DS to keep track of the S. Tree constructed.

for each node: we have this DS that has 4 COMP.

node n: **n. state**: in the state space to which n corresponds

n. Parent: the node that generated n.

n. Action: the action applied to the parent that lead to n.

n. PATHCOST: the cost, $g(n)$ of the path from the initial state to n. (as indicated by the parent pointers).

Easy to deduce given the parent node component.

③ implementation: function `childNode(parent, action, problem)` returns a node with:

n. state = `problem.result(parent.state, action)`

n. Parent = `parent`.

n. Action = `Action`.

n. PATHCOST = `parent.pathcost + problem.stepcost(parent.state, action)`.

Special case of Redundant paths (when there is more than one alternative path).

[We can use the definition of the problem to eliminate the redundant paths! from $n!$ to $1!$ (other)]

► In some cases, they are unavoidable! Where actions are Reversible.

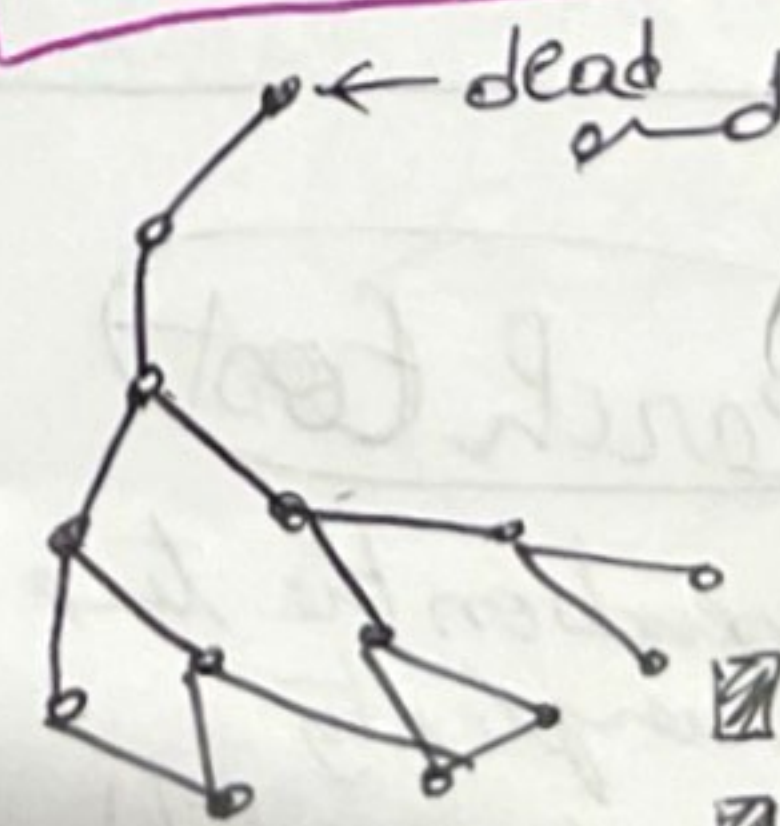
GRAPH-SEARCH

Better, augment the Tree-SEARCH with a DS called the explored set (hash table)

Which remembers every expanded node so as not to repeat expanding it!

→ every expanded node is added to the explored set

! DEAD ENDS: both its successors are already explored via other paths. (understand the slide 37)

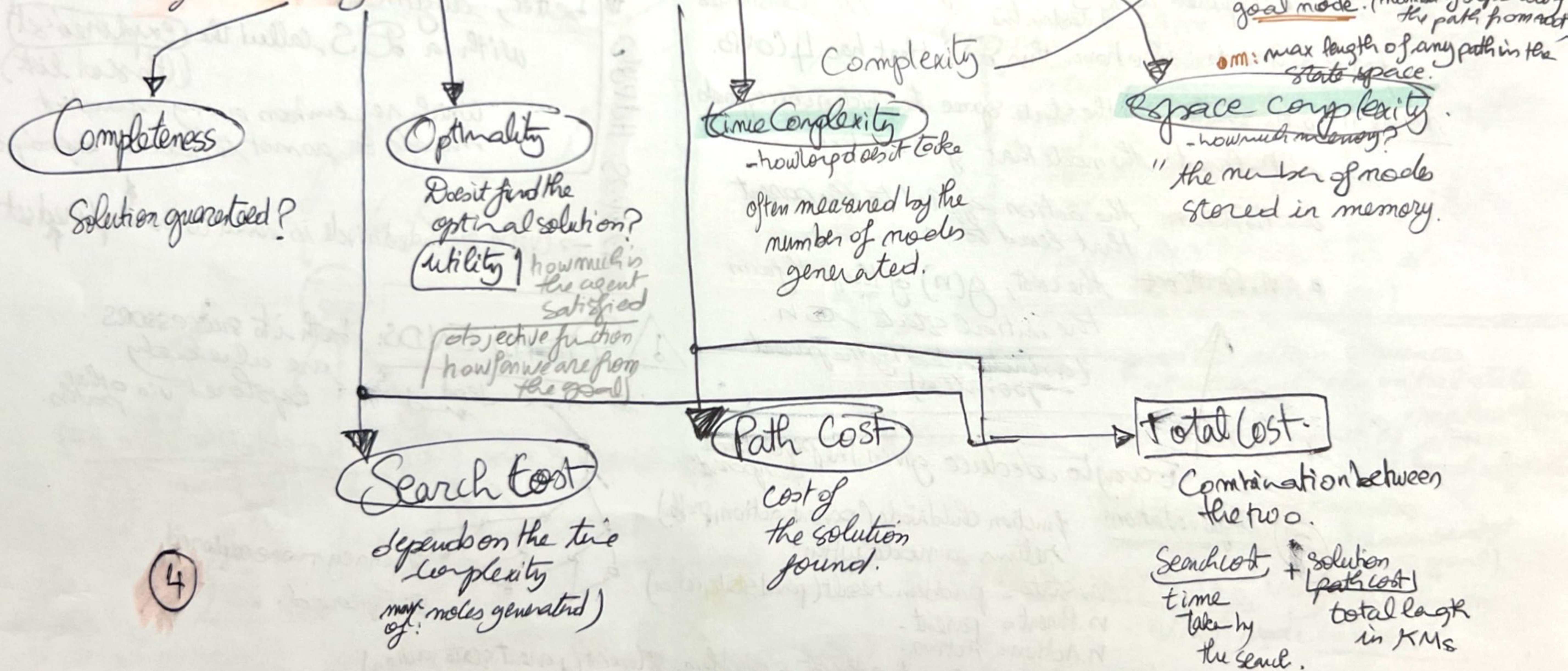


Grey more explored
Explored.

hash table for efficient checking!
 $O(1)$ insertion + lookup.

Remark: ensure that the in each DS (frontier/ES) the sets are in a Canonical form
 [logically equivalent states map to the same DS (node)]
 eg: bit vector representation, sorted list without repetition for states represented as sets are canonical!
 hash table must consider equality between states
 $\{B, U, V\} = \{U, V, B\}$

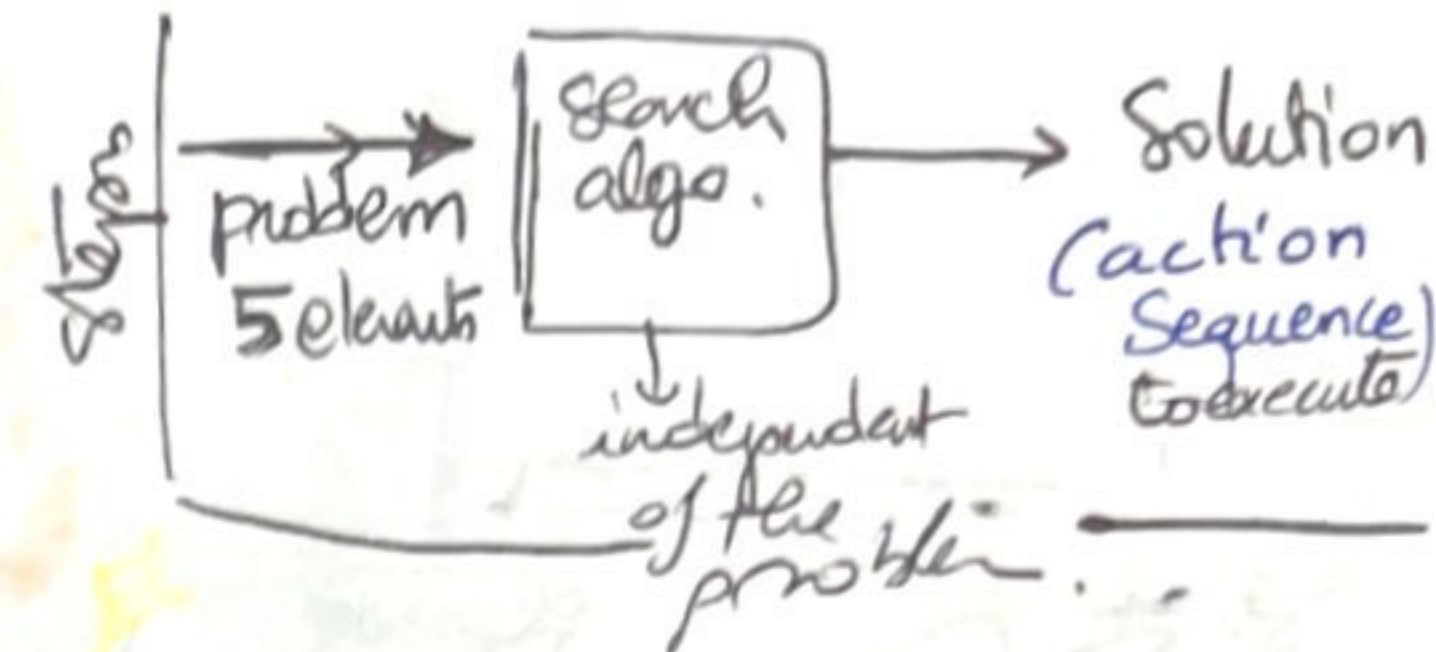
Measuring the Problem Solving performance



④

Chapter 03:

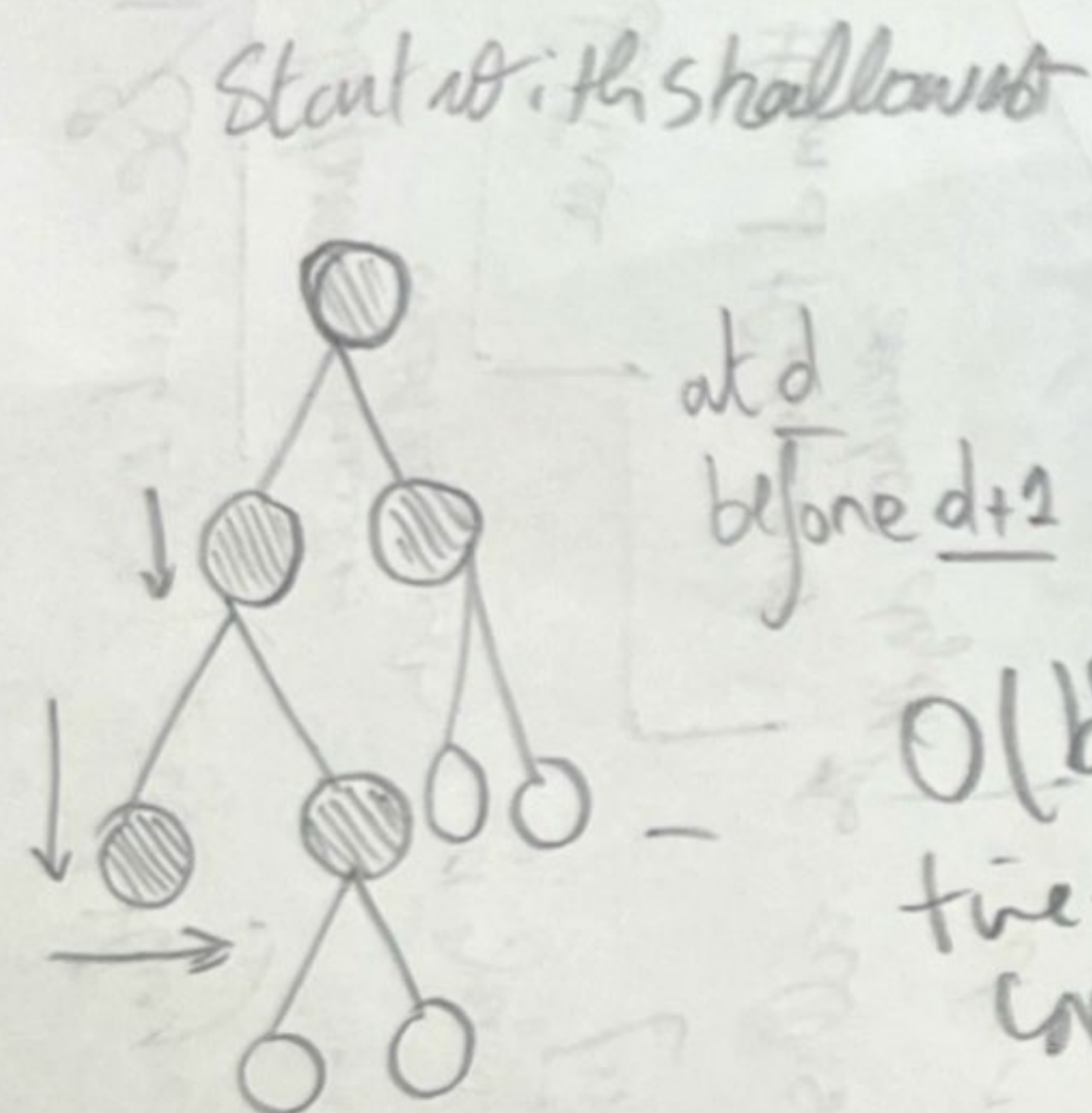
Search Algorithms



Uninformed Searches: Process of looking for a Sequence of actions to reach the GOAL

also called *blind search strategies*.

① Breadth-First-Search.



$O(b^d)$
time + space

② Uniform-Cost-Search

$O(b^{1+\frac{\epsilon}{\epsilon_0}})$
for cost $\epsilon > 0$
complete if $b < \infty$
Yes optimal.

③ Depth-First-Search

space	time
$O(bd)$	$O(b^d)$

④ Depth-Limited-Search

space	time
$O(b)$	$O(b^l)$

④ Bidirectional Search

$O(b^{d/2})$

⑤

